

PUBLIC TECHNICAL CASE STUDY

Debugging a Long-Run FreeRTOS Hang During Legacy RTOS Migration

A sanitized engineering investigation narrative for portfolio publication

Public disclosure boundary	
Scope	Public-facing technical case study. Company names, product names, project identifiers, protocol names, exact MCU part numbers, OS versions, private function names, logs, counters, and private implementation details are omitted or generalized.
Technology level	Cortex-M class microcontrollers, FreeRTOS, mutex blocking, interrupt masking, PendSV, PRIMASK, BASEPRI, and legacy RTOS compatibility behavior.
Disclosure choice	FreeRTOS is named because it is a public operating system and central to the technical mechanism. The previous commercial RTOS is described generically as the legacy RTOS unless a later review confirms that naming it is acceptable.

Executive Summary

A rare long-run hang appeared after migrating legacy embedded control software to a FreeRTOS-based Cortex-M platform. The visible stop point was deep inside RTOS scheduler list handling, which initially made the issue look like scheduler list corruption or a kernel-level failure. The investigation treated that visible failure as a symptom rather than the first cause.

The working hypothesis was that a potentially blocking recursive mutex acquisition was being reached while the caller was still inside an interrupt-disabled compatibility region inherited from the previous RTOS design. On FreeRTOS for Cortex-M, a contended mutex can require the current task to block and PendSV to perform the context switch. If that context switch is masked at the wrong moment, the blocking transition cannot complete normally.

The production fix was architectural: move potentially blocking RTOS operations outside interrupt-disabled regions, keep protected sections to short snapshot and commit phases, and add fail-fast guards so the unsafe pattern cannot silently return. No FreeRTOS kernel patch was required.

1. Key Facts

Visible symptom	The debugger stopped inside RTOS list insertion logic. A traversal pointer appeared not to advance, making the system look stuck in scheduler list handling.
Initial interpretation	Treat the list behavior as the place where the invalid scheduling state became

	visible, not as proof that the kernel itself was the original cause.
Hypothesis	A blocking mutex path was reached while the caller was still inside an interrupt-disabled compatibility region.
Mechanism under test	A contended mutex requires task blocking and a context switch. On Cortex-M FreeRTOS ports, PendSV is normally the exception that performs that switch.
Production decision	Keep interrupt-disabled regions short. Use them for snapshot and commit work only. Move blocking RTOS calls outside those regions.
Regression guard	Before any blocking wait, assert that the interrupt mask and compatibility-layer nesting state are clear.

2. Problem Observed

During long-run execution after the RTOS migration, the firmware eventually appeared hung. The debugger stopped inside RTOS list insertion code, which is part of scheduler and blocking-list management. The local symptom looked like a traversal pointer had become self-referential or was no longer advancing through the list.

That observation was important, but it was not enough to declare a FreeRTOS bug. Scheduler list code is often where corrupted task state, invalid blocking context, or illegal API usage finally becomes visible. The investigation therefore moved outward from the visible stop point and asked: what caller state could make a blocking transition unsafe?

3. Working Hypothesis

Hypothesis

A potentially blocking recursive mutex acquisition occurred while the caller was still inside a legacy interrupt-disabled region. If the mutex was unavailable, FreeRTOS needed to block the current task and run a context switch, but the required switch was masked by the caller state.

The suspected failing path was:

- The current task enters a legacy compatibility region that disables configurable interrupts.
- The task reaches a recursive mutex acquisition inside that region.
- The mutex is already owned by another task, so the current task cannot acquire it immediately.
- FreeRTOS must place the current task into a blocked state and request a context switch.
- The required context switch cannot run at that moment because the caller has masked it.

4. Pre-Check Evidence

A fail-fast guard was placed at the boundary where a non-blocking acquisition becomes a blocking acquisition. The guard was not the production fix by itself; it was evidence that the caller context was unsafe in the original flow and safe in the corrected flow.

Flow	Context at blocking boundary	Interpretation
Original structure	Blocking acquisition could be reached while the compatibility interrupt mask was still active.	Unsafe. The code path could ask the RTOS to block while scheduling was not fully available.
Corrected structure	Blocking acquisition was reached only after leaving the interrupt-disabled region.	Safe. The RTOS could block the current task and perform the required context switch.

5. FreeRTOS Mechanism

In the contended mutex path, FreeRTOS must move the current task into a blocked state and allow another ready task to run. On Cortex-M FreeRTOS ports, PendSV is normally used to perform the actual context switch. This matters because interrupt masking is not just an interrupt-latency detail; it can also affect whether the system can complete a blocking operation safely.

State or mechanism	Public explanation
PRIMASK	A Cortex-M interrupt mask that can block configurable exceptions. If active across a blocking RTOS call, it can prevent the required switch from running.
BASEPRI	A priority threshold used by FreeRTOS critical sections on Cortex-M. It is separate from a broad application-level PRIMASK-based mask.
PendSV	The exception FreeRTOS normally uses to perform a context switch after a task blocks or yields.
Mutex contention	The point where a mutex acquisition changes from a simple fast path into a blocking scheduling operation.

6. Why the Legacy Pattern Could Survive Before

The legacy firmware pattern came from an environment where interrupt-disabled regions and blocking synchronization had different scheduler semantics. In the previous RTOS design, the blocking path could publish wait state and enter a scheduler-aware switch sequence in a way that preserved the legacy expectation. After migration, the same source-level pattern no longer had the same runtime meaning.

The lesson is not that one RTOS is good and the other is bad. The lesson is that compatibility wrappers can preserve API names while changing the execution contract underneath. A pattern that was safe under one scheduler can become unsafe when the new kernel depends on a different context-switch mechanism.

7. Controlled Experiment

A controlled experiment was used to test the causal mechanism without patching the FreeRTOS kernel. The experiment preserved the original legacy call structure and changed only the compatibility-layer behavior around the suspected masked-blocking path.

1. First attempt a non-blocking mutex acquisition.

2. If the mutex is acquired immediately, return through the fast path because no blocking or context switch is required.
3. If the mutex is unavailable and the caller is in the suspected masked path, temporarily allow the required context switch in the experimental build.
4. Perform the blocking acquisition.
5. After the task resumes, restore the previous mask state.

Experiment-only behavior

This was not selected as the production design. Temporarily opening the interrupt mask inside a legacy critical region can prove the mechanism, but it is too implicit and too risky as a long-term architecture.

8. Experiment Result

The controlled build continued running through the test window. The important result was expressed qualitatively rather than with exact private test counters: every observed masked-blocking event resumed when the required context switch was allowed to complete, and the original list-insertion hang was not reproduced during the experiment.

That result supported the hypothesis that the visible scheduler-list hang was downstream of a masked context-switch problem, not an ordinary FreeRTOS kernel defect and not random memory corruption.

9. Production Fix

The production fix deliberately avoided changing the FreeRTOS kernel, broadly changing the interrupt-disable wrapper, or shipping the temporary experiment behavior. The final design corrected the architecture at the call-site pattern level.

Not selected	Production decision
Patch RTOS scheduler or list code.	Do not patch the kernel. The scheduler was where the symptom appeared, not the first cause.
Change the global interrupt-disable wrapper broadly.	Do not change global masking semantics without a larger compatibility design review.
Ship the temporary unmask experiment.	Use the experiment only as causal proof, not as production behavior.
Keep the legacy blocking call inside the disabled region.	Move potentially blocking RTOS calls outside disabled regions. Keep protected work short.

Conceptually, the corrected structure became:

```

disable_interrupt();
/* short snapshot / protected state read */
enable_interrupt();

take_mutex_blocking();
/* potentially blocking work occurs while scheduling is available */
release_mutex();

disable_interrupt();
/* short commit / protected state update */
enable_interrupt();

```

10. Regression Guard

A guard was kept at the transition from fast path to blocking path. The intent is to fail early during development if future code accidentally reintroduces the unsafe pattern.

- Try the recursive mutex acquisition with zero timeout first.
- If it succeeds immediately, return; no task blocking is required.
- If blocking is required, assert that the relevant interrupt mask and compatibility nesting state are clear.
- Only then perform the blocking wait.

The guard is not a replacement for correct architecture. It is a regression detector that protects the architecture from silently drifting back into the old unsafe shape.

11. Final Rule

Final engineering rule

Never perform a potentially blocking RTOS operation while the required context switch is masked.

For migration projects, the broader rule is to audit legacy critical-section patterns before assuming that compatibility wrappers preserve behavior. API compatibility is not the same thing as scheduler-semantics compatibility.

12. Public Version Notes

This document intentionally names FreeRTOS because the technical mechanism depends on public Cortex-M FreeRTOS behavior and the relevant concepts are widely documented: mutex blocking, PendSV context switching, PRIMASK, and BASEPRI. The previous RTOS, product context, project identifiers, exact versions, source names, counters, and hardware identifiers are generalized because they are not necessary to explain the engineering lesson.

If this case study is later published as a web page, it should use the same disclosure boundary and should not include private screenshots, private PDFs, source excerpts from non-public repositories, exact logs, or project-specific names.